# (S2) Sockets Programming: Client and Server

Ion Mandoiu
Laurent Michel
Revised by M. Khan, J. Shi and W. Wei

# Review

- TCP/IP
  - Name/IP and port number

- Socket
  - We get a file descriptor and we can send and recv
    - Place send() and recv() in a loop!

# A Client-Server World

Establish a connection between a server and a client

- Server
  - Listens on a port for inbound “let’s talk” requests
  - Can talk to many clients at the same time
- Client
  - Initiates the communication with the server
  - Must know
    - Server name (which is used to find the IP address)
    - Protocol to use (TCP or UDP)
    - Port number

# Server side (TCP)

- After creating a socket
  - **bind()** Server **binds** (reserves) a port for chatting
  - **listen()** Server **listens** on port for “Let’s talk” requests
  - **accept()** When a request comes in…
    - Server allocates a **separate socket** to **accept** private communication with the client (same host, different port)
      - The original socket remains available for future “Let’s talk”
      - The new socket is bound to an ephemeral port
    - Communication goes on on separate sockets with
      - **send() / recv()**
  - **close()** Conversation ends via a close

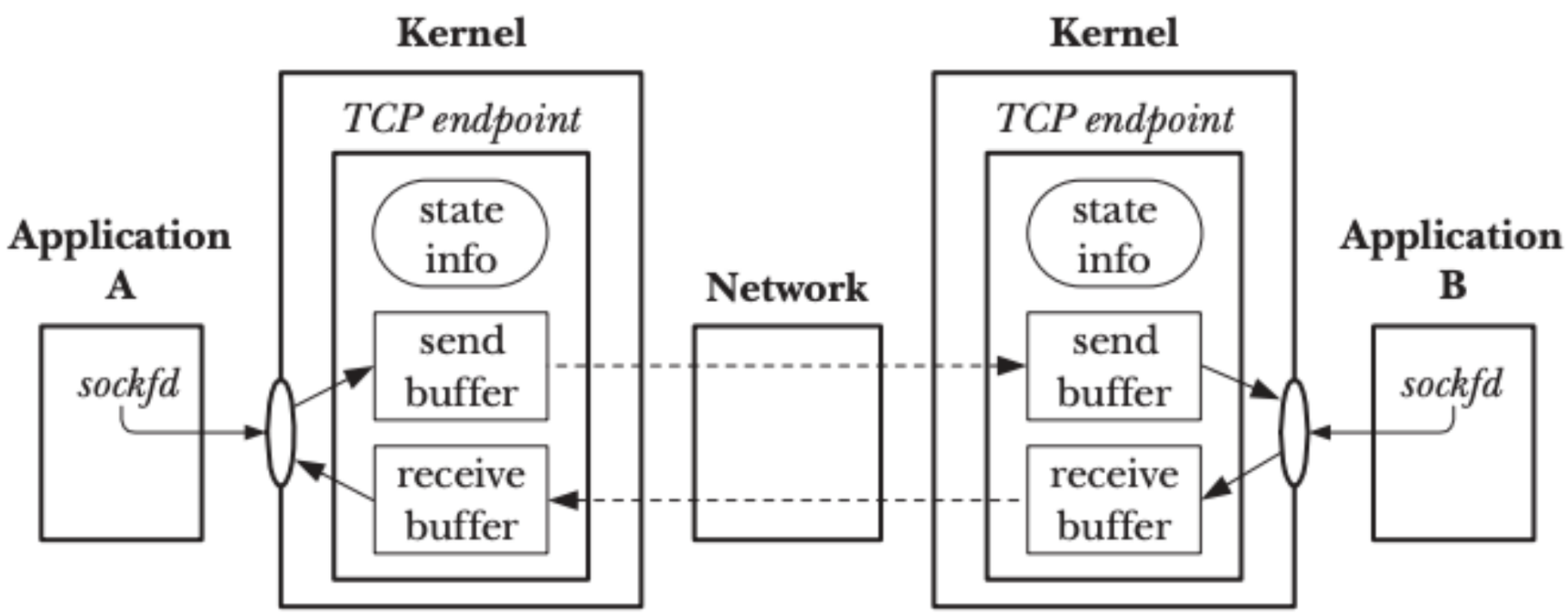



**Figure 58-8:** Connected TCP sockets

# Analogy Between Sockets and Telephone

| | Sockets | Telephone |
|---|---|---|
| socket() | Create a socket | Buy a telephone |
| bind() | Bind to a port | Plug in/Insert a Sim |
| listen() | Get ready to respond | Get ready for ringtone |
| accept() | Accept requests | Pick up the phone |
| send()/recv() | Communication | Listen and talk (at an extension) |
| close() | Close the connection | Hang up |

# Client side (TCP)

- After creating a socket
  - **connect()** Client initiates a session
    - Need server name, type, and port
  - Then, communication goes on with **send() / recv()**
  - **close()** Client closes a session

Note that no bind() is needed on the client side

OS allocates a port

# Getting back to server …

```
#include <sys/types.h>
#include <sys/socket.h>

int bind(   int sockfd,
            const struct sockaddr *addr,
            socklen_t addrlen);
```

- sockfd is the socket file descriptor. It does not have an address!
- What is struct sockaddr?
  - It holds address information for many types of socket
  - That is why there is a separate addrlen to indicate its size

# Socket address types

```
// generic socket address
struct sockaddr {
  unsigned short    sa_family;    // address family, AF_xxx
  char       sa_data[14];  // 14 byte protocol address. Place holder
};

// we are using Internet socket address
struct sockaddr_in sa;       // socket address IPv4
struct sockaddr_in6 sa6;    // socket address IPv6
// struct sockaddr_in * can be cast to struct sockaddr *
// Do I really need to learn how to set the fields in sa and sa6?
```

# addrinfo structure

```
// A recent invention for preparing the socket address
struct addrinfo {
    int        ai_flags;     // AI_PASSIVE, AI_CANONNAME, etc.
    int        ai_family;    // AF_INET, AF_INET6, AF_UNSPEC
    int        ai_socktype;  // SOCK_STREAM, SOCK_DGRAM
    int        ai_protocol;  // use 0 for "any"
    size_t     ai_addrlen;   // size of ai_addr in bytes
    struct sockaddr *ai_addr;      // struct sockaddr_in or _in6
    char       *ai_canonname; // full canonical hostname
    struct addrinfo *ai_next;      // linked list, next node
};
```

# getaddrinfo()

```
int getaddrinfo(const char *node,     // "www.example.com" or IP
                const char *service,  // "http" or port #
                const struct addrinfo *hints,
                struct addrinfo **res);
```

- This function finds addrinfo for you!
  - Provide you a linked-list, *res*. A list of results!
- *node* is the host name to connect to, or an IP address.
- *service* can be a port number, like “80”, or the name of a particular service  like “http” or “ftp” or “telnet” or “smtp”, etc.
- *hints* points to a struct addrinfo that you've already filled out with relevant information.

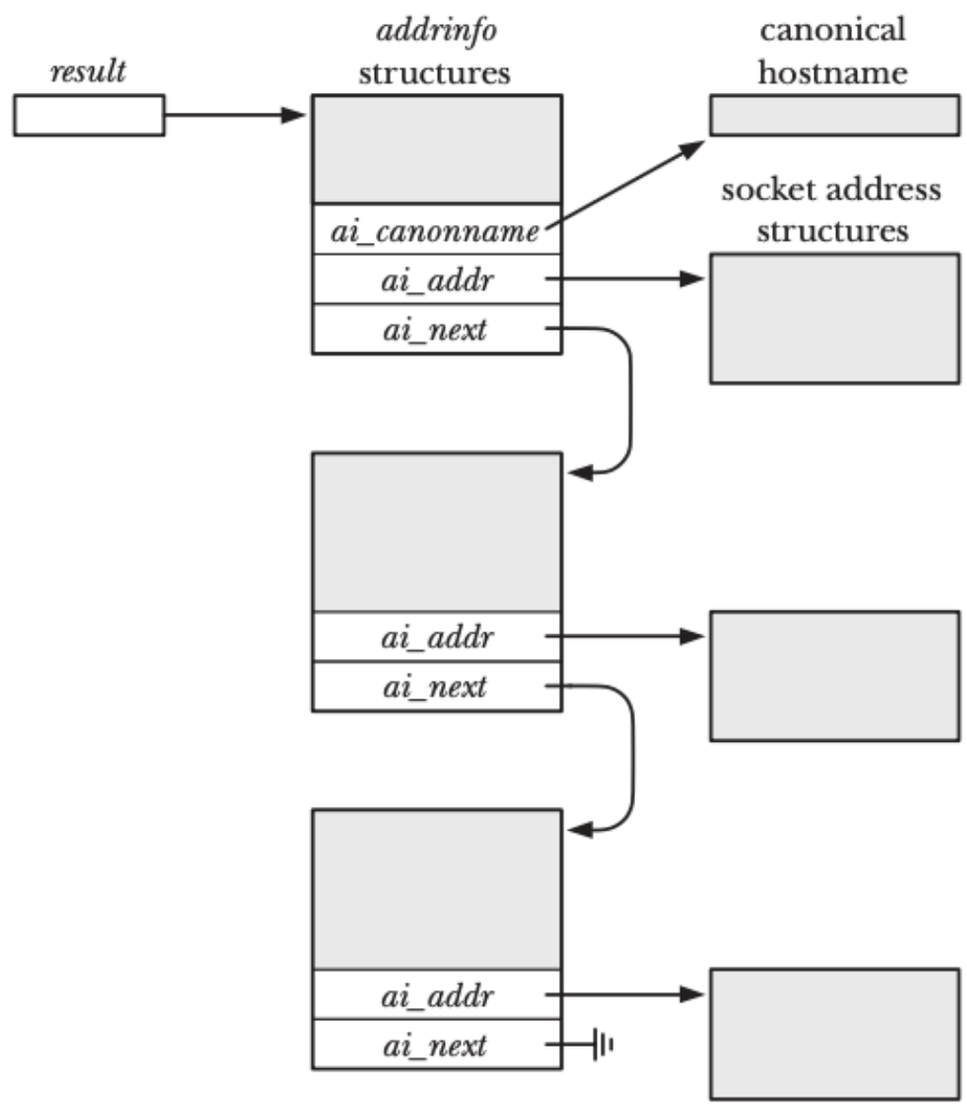



**Figure 59-3:** Structures allocated and returned by *getaddrinfo()*

# Usage of getaddrinfo() for server

```
struct addrinfo hints, *servinfo; // res will point to the results


memset(&hints, 0, sizeof hints); // make sure the struct is empty
hints.ai_family = AF_UNSPEC;     // don't care IPv4 or IPv6
hints.ai_socktype = SOCK_STREAM; // TCP stream sockets
hints.ai_flags = AI_PASSIVE;     // fill in my IP for me

if (getaddrinfo(NULL, "3490", &hints, &servinfo) != 0)
    die("Error");
// servinfo now points to a linked list of 1 or more struct addrinfos

// continue on the next page
```

# Now, let us create socket and then bind!

```
int sockfd, yes = 1;
struct addrinfo *p;
// use socket(); setsockopt(); bind(); to try each address on the list
// continue if any of them fails. Check return values!
for (p = servinfo; p != NULL; p = p->ai_next) {
sockfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(int));
// bind it to the port we passed in to getaddrinfo():
bind(sockfd, p->ai_addr, p->ai_addrlen);
break; // we have an usable address, stop trying
}
freeaddrinfo(servinfo);        // free the linked-list
```

# After bind()

```
// listen on that socket for "Let's talk" message.
status = listen(sockfd, 10);
checkError(status);
// client's address information, if you are interested in
struct sockaddr_storage client_addr;
while(1) {
   socklen_t clientSize = sizeof(client_addr);
   // A new socket file descriptor is returned for chatting
   int chatSocket = accept(sockfd,
                           (struct sockaddr*)&client_addr,
                           &clientSize);
   checkError(chatSocket);
   // See demo code on how to display client address
   printf("We accepted a socket: %d\n",chatSocket);
   close(chatSocket);
}
```

# Usage of getaddrinfo() for client

```
struct addrinfo hints, *servinfo; // will point to the results


memset(&hints, 0, sizeof hints); // make sure the struct is empty
hints.ai_family = AF_UNSPEC;     // don't care IPv4 or IPv6
hints.ai_socktype = SOCK_STREAM; // TCP stream sockets
// get ready to connect
if (getaddrinfo("www.uconn.edu", "3490", &hints, &servinfo) != 0)
    die("Error");
// servinfo now points to a linked list of 1 or more struct addrinfos


// continue on the next page
```

# connect at the client side

```
int sockfd, yes = 1;
struct addrinfo *p;
// use socket(); connect(); to try each address on the list
// continue if any of them fails. Check return values!
for (p = servinfo; p != NULL; p = p->ai_next) {
sockfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
connect(sockfd, p->ai_addr, p->ai_addrlen);
// we have a connection if both are successful. Stop trying.
break;
}
…
freeaddrinfo(servinfo);        // free the linked-list
```

# Blocking and Talking to Many Clients

- On the server side
  - Accept is blocking (until a “let’s talk” request comes in)
  - If serving the client directly….
    - Inbound “Let’s talk” are queue up (up to maximum)
  - If the server wishes to get back to accepting:
    fork a process! or create a thread!
    - All real communication happens on the accepted socket
- On the client side
  - Connect is blocking (until the server responds or refuses)

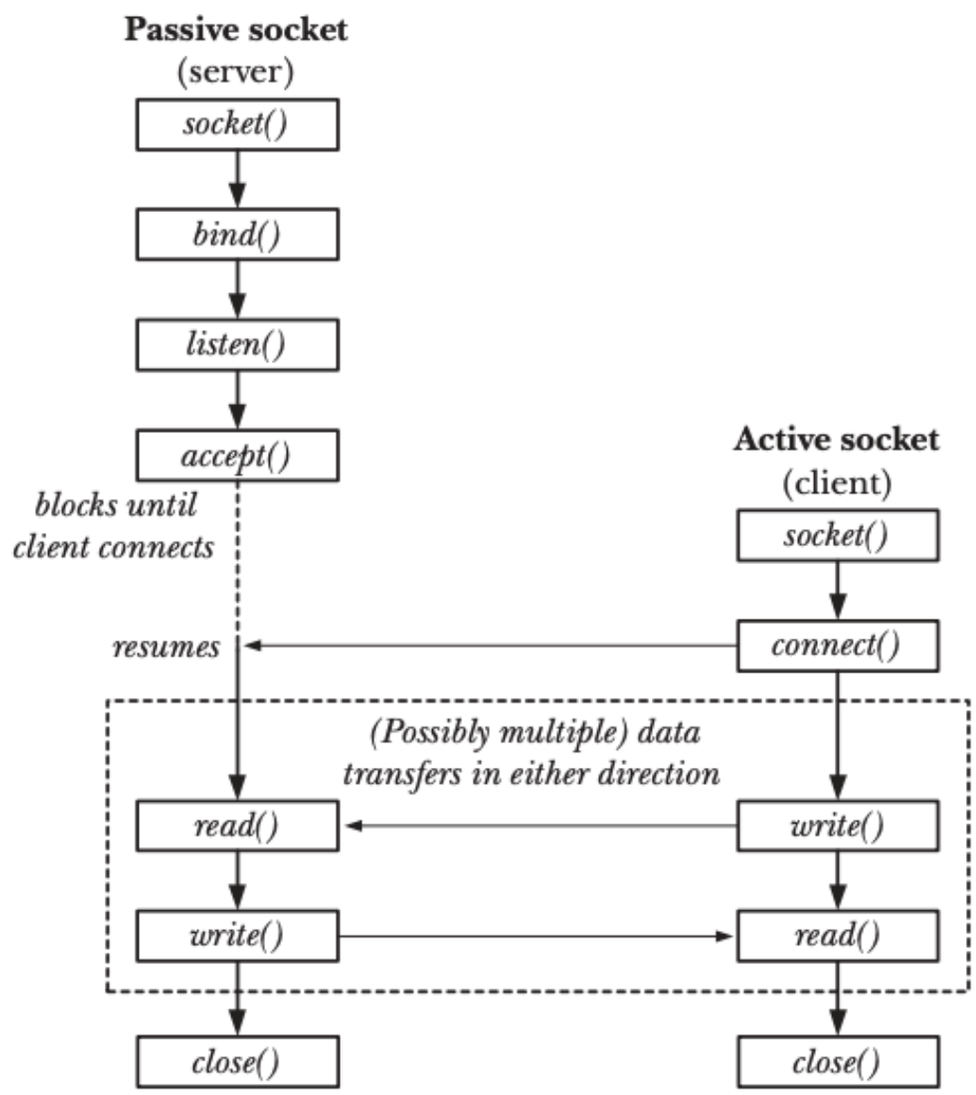
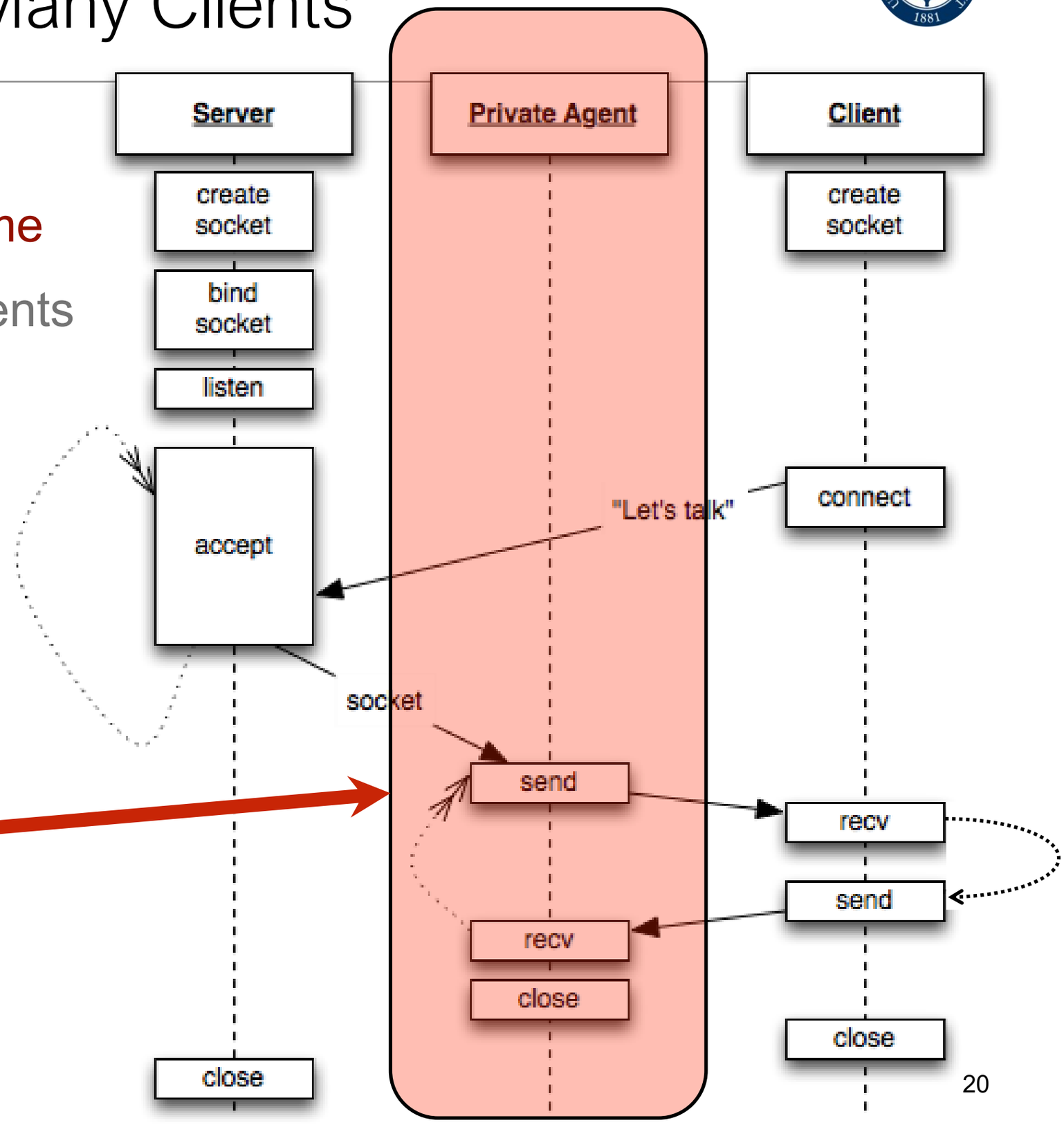



**Figure 56-1:** Overview of system calls used with stream sockets

# Server Dealing with Many Clients

- The server cannot talk to many clients at the same time
  - Let others to deal with clients
  - It can wait for new clients




**Could be a cloned process (fork!) … or a thread**

**or use functions like select()**